

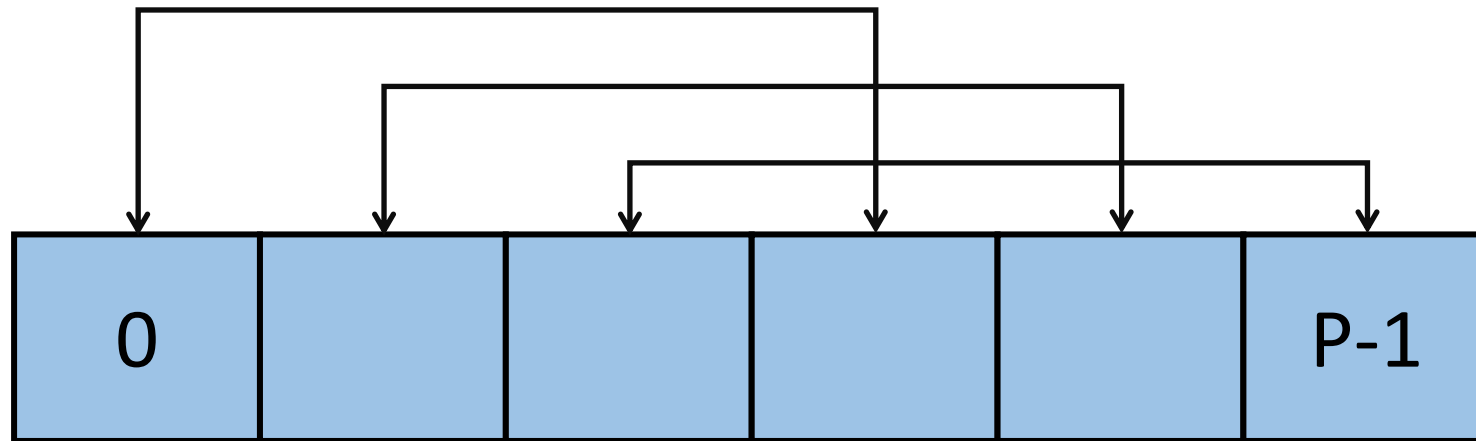
Communication Patterns

Lecture 7

January 28, 2026

Send-Recv Example ($2 * P/2$ pairwise communications)

Figure shows the **communication pattern**

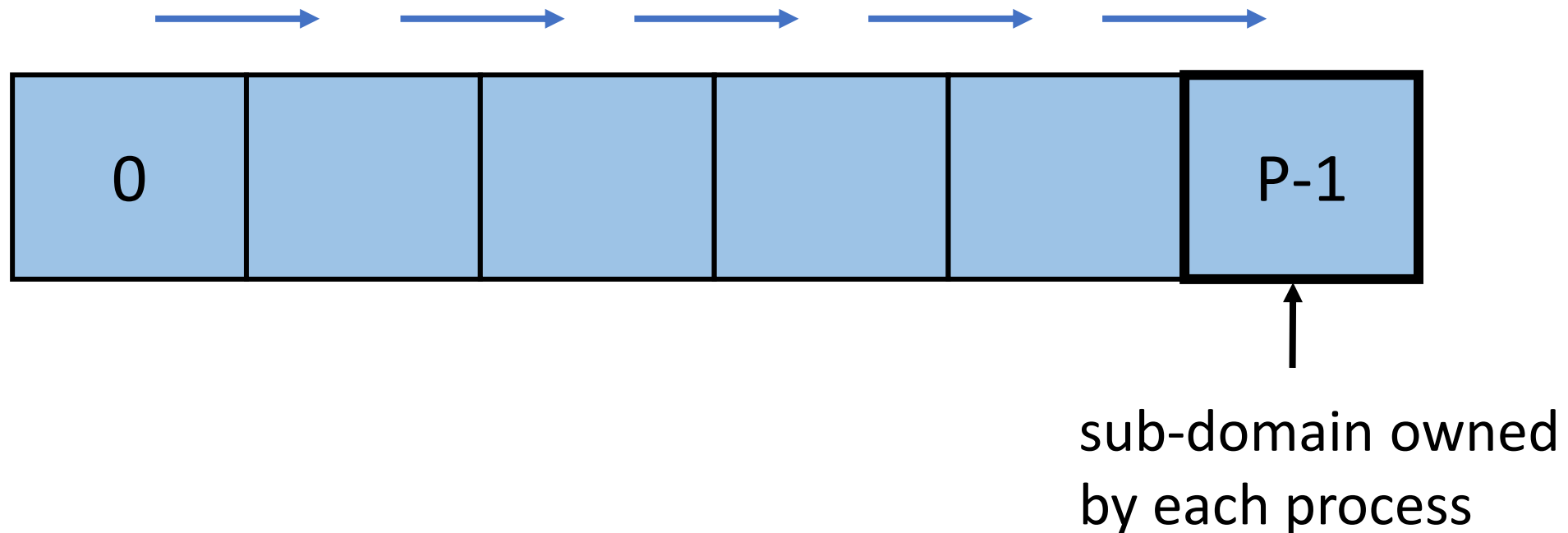


- How many destinations does each process have?
- How many senders does each process have?
- A process communicates with which process?

Code (one of the versions)

```
if (myrank < P/2) {  
    MPI_Send (data, count, MPI_INT, myrank+P/2, myrank+P/2,  
MPI_COMM_WORLD);  
    MPI_Recv (recvbuf, count, MPI_INT, myrank+P/2, myrank, MPI_COMM_WORLD,  
&status);  
}  
else {  
    MPI_Recv (recvbuf, count, MPI_INT, myrank-P/2, myrank, MPI_COMM_WORLD,  
&status);  
    MPI_Send (data, count, MPI_INT, myrank-P/2, myrank-P/2, MPI_COMM_WORLD);  
}
```

Nearest Neighbor (NN) 1D Exchange

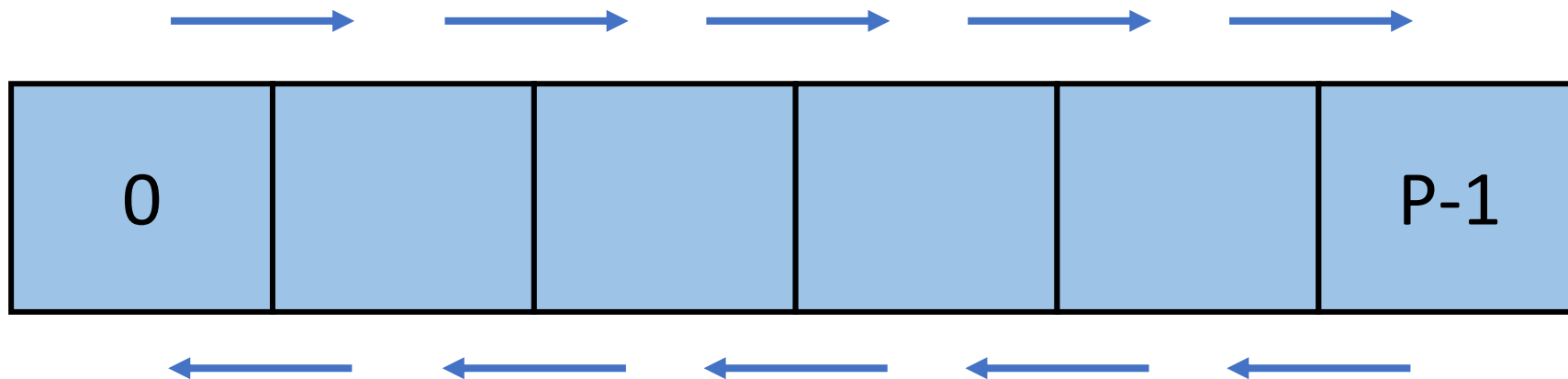


- 1D domain equally sub-divided into P processes
- Blue arrows show the **communication pattern**

Pseudocode

```
if (myrank < P-1) {  
    // Send/recv right neighbor  
    MPI_Send (data, count, MPI_DOUBLE, myrank+1, myrank+1,  
MPI_COMM_WORLD);  
}  
if (myrank > 0) {  
    // Send/recv left neighbor  
    MPI_Recv (recvbuf, count, MPI_DOUBLE, myrank-1, myrank,  
MPI_COMM_WORLD, &status);  
    printf ("%d received from %d\n", myrank, myrank-1);  
}
```

Nearest Neighbor (NN) 2-way 1D Exchange



- 1D domain equally sub-divided into P processes
- Blue arrows show the 2D **communication pattern**

Nearest Neighbor Pseudocode

Option 1: Schedule **right sends** followed by left sends

```
if (myrank < P-1)
{
    // Send/recv right neighbor
    MPI_Send (data, myArraySize, MPI_DOUBLE, myrank+1, myrank+1,
MPI_COMM_WORLD);
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, myrank+1, myrank,
MPI_COMM_WORLD, &status);
}
```



Tags?
Performance?

Nearest Neighbor Pseudocode

Option 1: Schedule right sends followed **by left sends**

```
if (myrank > 0)
{
    // Send/recv left neighbor
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, myrank-1, myrank,
MPI_COMM_WORLD, &status);
    MPI_Send (data, myArraySize, MPI_DOUBLE, myrank-1, myrank-1,
MPI_COMM_WORLD);
}
```



Tags?
Performance?

Output (1D NN exchange)

```
class > mpirun -np 4 ./nn1D-1 10000000
3 received from 2
2 received from 1
1 received from 0
0.003885
class > mpirun -np 4 ./nn1D-1 100000000
3 received from 2
2 received from 1
0.054378
1 received from 0
class > mpirun -np 4 ./nn1D-1 1000000000
3 received from 2
2 received from 1
0.340451
1 received from 0
```

Output (2-way NN)

```
2. pmalakar@172.27.19.10 3. pmalakar@172.27.19.2
CS633 mpirun -np 12 -hosts csews3,csews4,csews15 ./a.out 1000000
10 sent to 11
11 received from 10
9 sent to 10
10 received from 9
8 sent to 9
9 received from 8
7 sent to 8
8 received from 7
6 sent to 7
7 received from 6
5 sent to 6
6 received from 5
4 sent to 5
5 received from 4
3 sent to 4
4 received from 3
2 sent to 3
3 received from 2
1 sent to 2
2 received from 1
0 sent to 1
1 received from 0
1.586693

2. pmalakar@172.27.19.10
CS633 mpirun -np 6 -hosts csews
4 sent to 5
5 received from 4
3 sent to 4
4 received from 3
2 sent to 3
3 received from 2
1 sent to 2
2 received from 1
0 sent to 1
1 received from 0
0.733342
CS633
```

```
2. pmalakar@172.27.19.10 3. pmalakar@172.27.19.2
CS633 mpirun -np 20 -hosts csews10,csews4,csews15 ./a.out 1000000
18 sent to 19
19 received from 18
17 sent to 18
18 received from 17
16 sent to 17
17 received from 16
15 sent to 16
16 received from 15
14 sent to 15
15 received from 14
13 sent to 14
14 received from 13
12 sent to 13
13 received from 12
11 sent to 12
12 received from 11
10 sent to 11
11 received from 10
9 sent to 10
10 received from 9
8 sent to 9
9 received from 8
7 sent to 8
8 received from 7
6 sent to 7
7 received from 6
5 sent to 6
6 received from 5
4 sent to 5
5 received from 4
3 sent to 4
4 received from 3
2 sent to 3
3 received from 2
1 sent to 2
2 received from 1
0 sent to 1
1 received from 0
2.753074
CS633
```

Nearest Neighbor Pseudocode

Option 2: Schedule odd and even ranks alternately

```
if (?) {  
  
    MPI_Send (data, myArraySize, MPI_DOUBLE, ?, ?, MPI_COMM_WORLD);  
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, ?, ?, MPI_COMM_WORLD, &status);  
}  
  
else if (?) {  
  
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, ?, ?, MPI_COMM_WORLD, &status);  
    MPI_Send (data, myArraySize, MPI_DOUBLE, ?, ?, MPI_COMM_WORLD);  
}
```

Nearest Neighbor Pseudocode

Option 2: Schedule odd and even ranks alternately

```
if (myrank % 2 == 0 && myrank < P-1) {  
    // Send/recv right neighbour from even ranks  
    MPI_Send (data, myArraySize, MPI_DOUBLE, myrank+1, myrank+1, MPI_COMM_WORLD);  
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, myrank+1, myrank, MPI_COMM_WORLD, &status);  
}  
else if (myrank % 2 != 0 && myrank > 0) {  
    // Send/recv left neighbor  
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, myrank-1, myrank, MPI_COMM_WORLD, &status);  
    MPI_Send (data, myArraySize, MPI_DOUBLE, myrank-1, myrank-1, MPI_COMM_WORLD);  
}
```

Nearest Neighbor Pseudocode

Option 2: Schedule odd and even ranks alternately

```
if (?)
{
    // Send/recv right neighbor from odd ranks
    MPI_Send (data, myArraySize, MPI_DOUBLE, ?, ?, MPI_COMM_WORLD);
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, ?, ?, MPI_COMM_WORLD, &status);
}

else if (?)
{
    // Send/recv left neighbor
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, ?, ?, MPI_COMM_WORLD, &status);
    MPI_Send (data, myArraySize, MPI_DOUBLE, ?, ?, MPI_COMM_WORLD);
}
```

Nearest Neighbor Pseudocode

Option 2: Schedule odd and even ranks alternately

```
if (myrank % 2 != 0 && myrank < P-1)
{
    // Send/recv right neighbor from odd ranks
    MPI_Send (data, myArraySize, MPI_DOUBLE, myrank+1, myrank+1, MPI_COMM_WORLD);
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, myrank+1, myrank, MPI_COMM_WORLD, &status);
}

else if (myrank % 2 == 0 && myrank > 0)
{
    // Send/recv left neighbor
    MPI_Recv (recvbuf, myArraySize, MPI_DOUBLE, myrank-1, myrank, MPI_COMM_WORLD, &status);
    MPI_Send (data, myArraySize, MPI_DOUBLE, myrank-1, myrank-1, MPI_COMM_WORLD);
}
```

Single Host csews* (Option 1 vs. 2)

```
for i in `seq 1 5` ; do mpirun -np 4 ./nn-1 1000000 ; done
```

0.006751

0.006896

0.006518

0.006310

0.006356

```
for i in `seq 1 5` ; do mpirun -np 4 ./nn-2 1000000 ; done
```

0.006183

0.017730

0.006718

0.006862

0.006701

Single Host project* (Option 1 vs. 2)

```
class$ mpirun -np 8 ./nn-1 1000000
```

```
0.037653
```

```
class$ mpirun -np 8 ./nn-2 1000000
```

```
0.046915
```

```
class$ mpirun -np 8 ./nn-1 100000000
```

```
6.498764
```

```
class$ mpirun -np 8 ./nn-2 100000000
```

```
3.094525
```

```
class$ mpirun -np 8 ./nn-1 1000000
```

```
0.059250
```

```
class$ mpirun -np 8 ./nn-2 1000000
```

```
0.017978
```

```
class$ mpirun -np 8 ./nn-1 100000000
```

```
6.279696
```

```
class$ mpirun -np 8 ./nn-2 100000000
```

```
3.531164
```


Two Hosts (Option 1 vs. 2)

```
for i in `seq 1 5` ; do mpirun -np 4 -hosts csews1,csews10 ./nn-1 1000000 ; done
```

0.450281

0.426031

0.419316

0.445110

0.416786

```
or i in `seq 1 5` ; do mpirun -np 4 -hosts csews1,csews10 ./nn-2 1000000 ; done
```

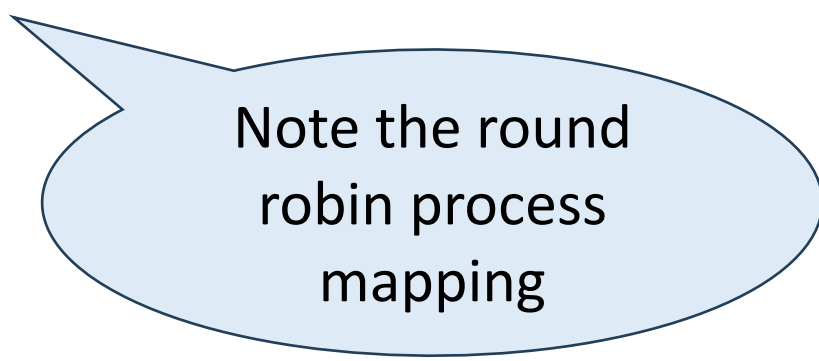
0.405743

0.423926

0.410813

0.420823

0.430066



Note the round
robin process
mapping

Three Hosts (Option 1 vs. 2) Run 1

```
class > mpirun -np 8 ./nn-1 100000  
0.002694  
class > mpirun -np 8 ./nn-2 100000  
0.002872  
class > mpirun -np 8 -hosts csews1,csews2,csews5 ./nn-1 100000  
0.110686  
class > mpirun -np 8 -hosts csews1,csews2,csews5 ./nn-2 100000  
0.060402  
class > mpirun -np 8 -hosts csews1,csews2,csews5 ./nn-1 1000000  
0.973480  
class > mpirun -np 8 -hosts csews1,csews2,csews5 ./nn-2 1000000  
0.634143
```

Three Hosts (Option 1 vs. 2) Run 2

```
class > mpirun -np 8 ./nn-1 100000  
0.003966  
class > mpirun -np 8 ./nn-2 100000  
0.003265  
class > mpirun -np 8 -hosts csews1,csews2,csews5 ./nn-1 100000  
0.110145  
class > mpirun -np 8 -hosts csews1,csews2,csews5 ./nn-2 100000  
0.063298  
class > mpirun -np 8 -hosts csews1,csews2,csews5 ./nn-1 1000000  
0.971313  
class > mpirun -np 8 -hosts csews1,csews2,csews5 ./nn-2 1000000  
0.552195
```

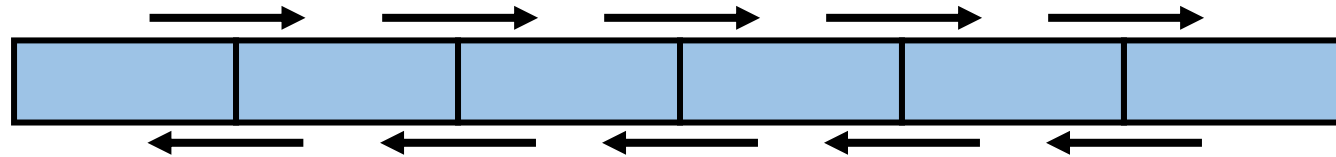
Timing Option 1 vs. Option 2

```
CS633 for i in `seq 1 5` ; do mpirun -np 20 -hosts csews1,csews5,csews10,csews15,csews22 ./nn-1 1000 ; done
0.017778
0.018780
0.013771
0.014954
0.017533
CS633 for i in `seq 1 5` ; do mpirun -np 20 -hosts csews1,csews5,csews10,csews15,csews22 ./nn-2 1000 ; done
0.010514
0.006946
0.007929
0.013788
0.014448
CS633 █
```

Timing Option 1 vs. Option 2

```
CS633
CS633 for i in `seq 1 5` ; do mpirun -np 20 -hosts csews1,csews5,csews10,csews15,csews22 ./nn-1 1000000 ; done
11.241324
11.259576
11.257508
11.244096
11.249499
CS633 for i in `seq 1 5` ; do mpirun -np 20 -hosts csews1,csews5,csews10,csews15,csews22 ./nn-2 1000000 ; done
5.506413
5.591499
5.390126
5.447412
5.908334
```

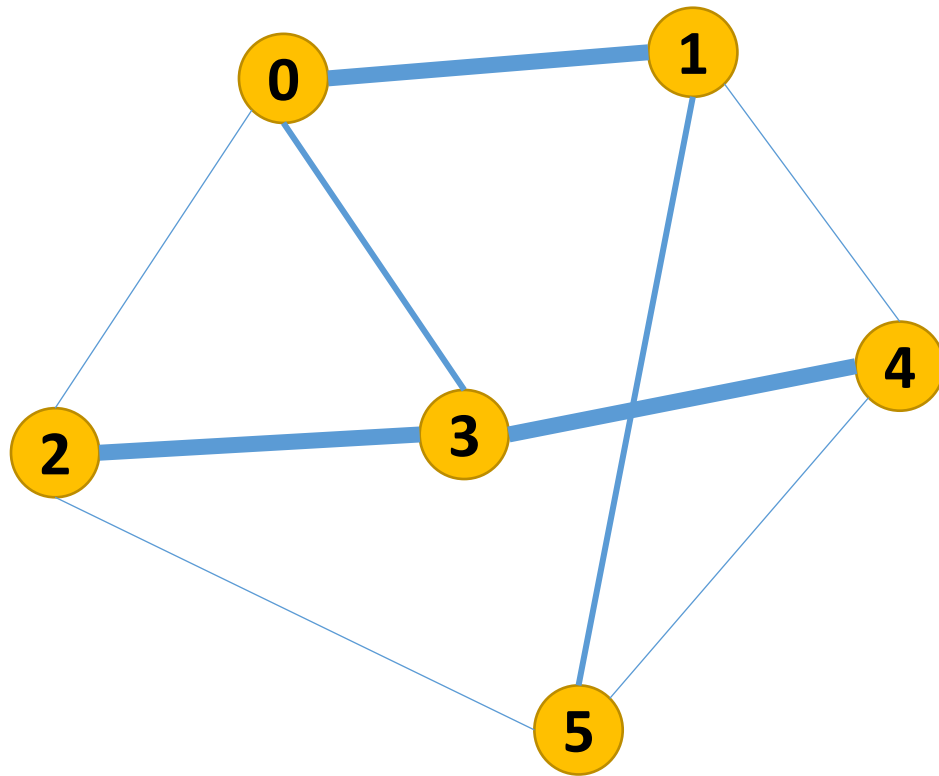
Timing NN (Option 1)



```
class mpirun -np 8 ./nn-1 1000000
0.014654
class mpirun -np 8 -hosts csews1:4,csews2:4 ./nn-1 1000000
0.199396
class mpirun -np 16 -hosts csews1:8,csews2:8 ./nn-1 1000000
0.200583
class mpirun -np 16 -hosts csews1:6,csews2:6,csews3:6 ./nn-1 1000000
0.285543
class mpirun -np 32 -hosts csews1:8,csews2:8,csews3:8,csews4:8 ./nn-1 1000000
0.489656
```

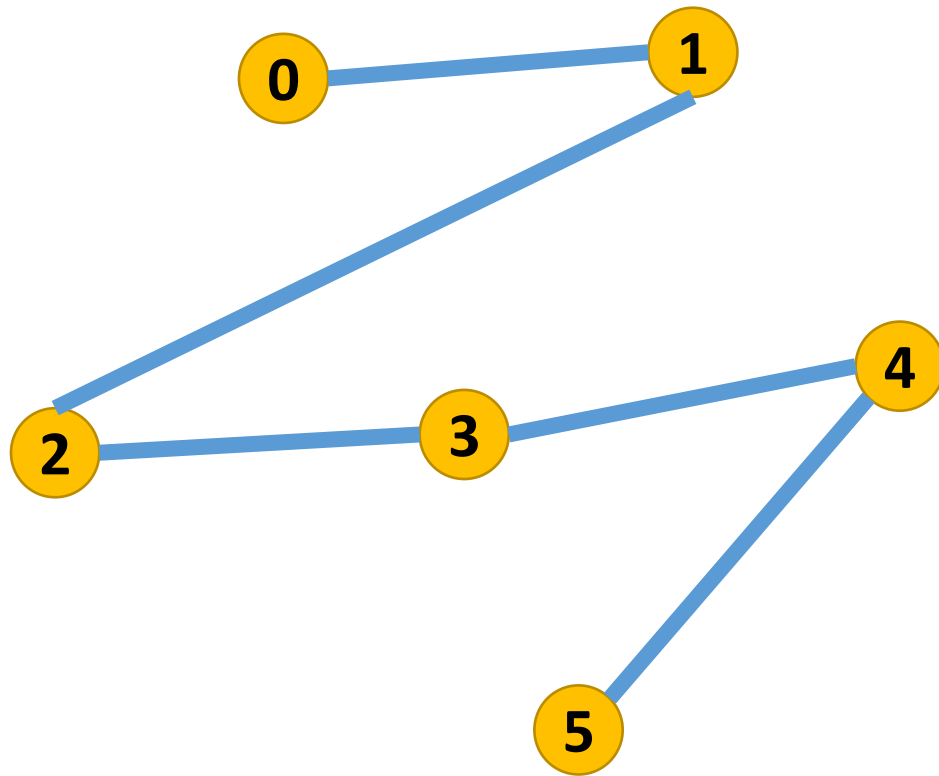
Communication Graph/Matrix

64, 256, 512 bytes (lighter to darker shade)



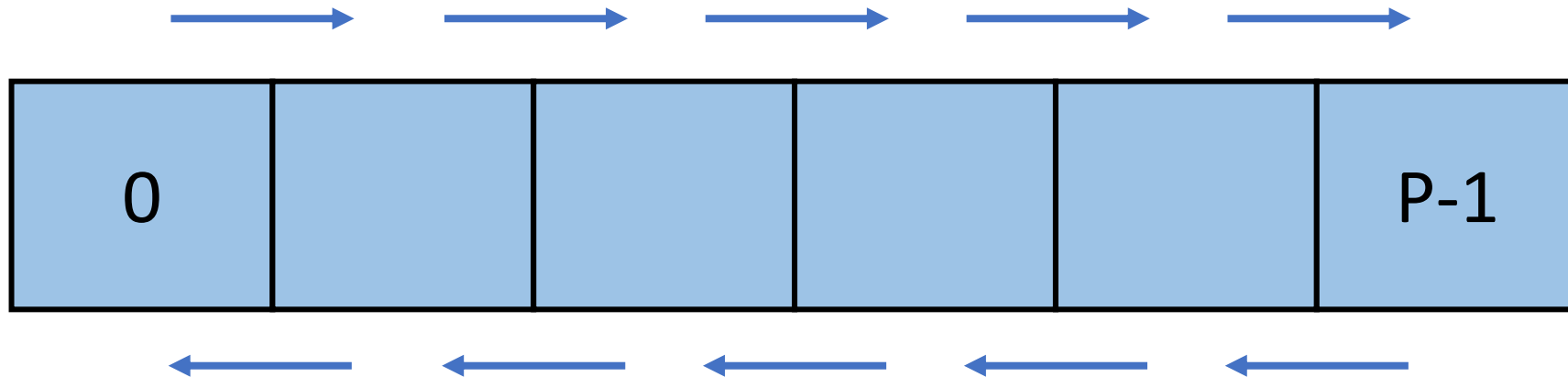
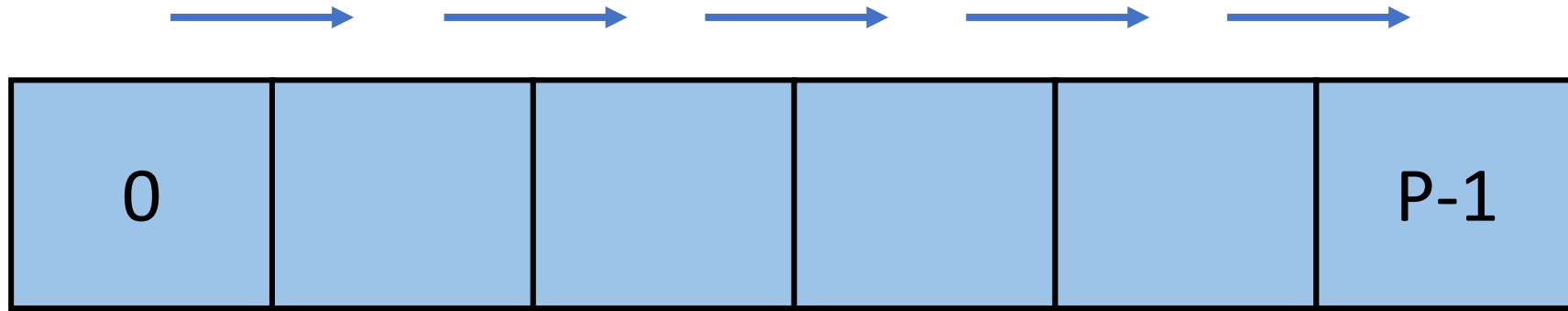
	512	64	256		
512				64	256
64			512		64
256		512		512	
	64		512		64
	256	64		64	

Communication Graph/Matrix for 2-way NN

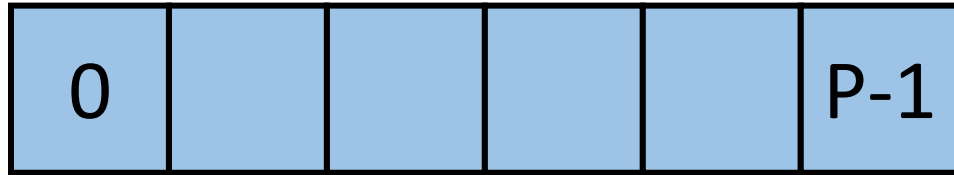


	0	1	2	3	4	5
0		512				
1	512		512			
2		512		512		
3			512		512	
4				512		512
5					512	

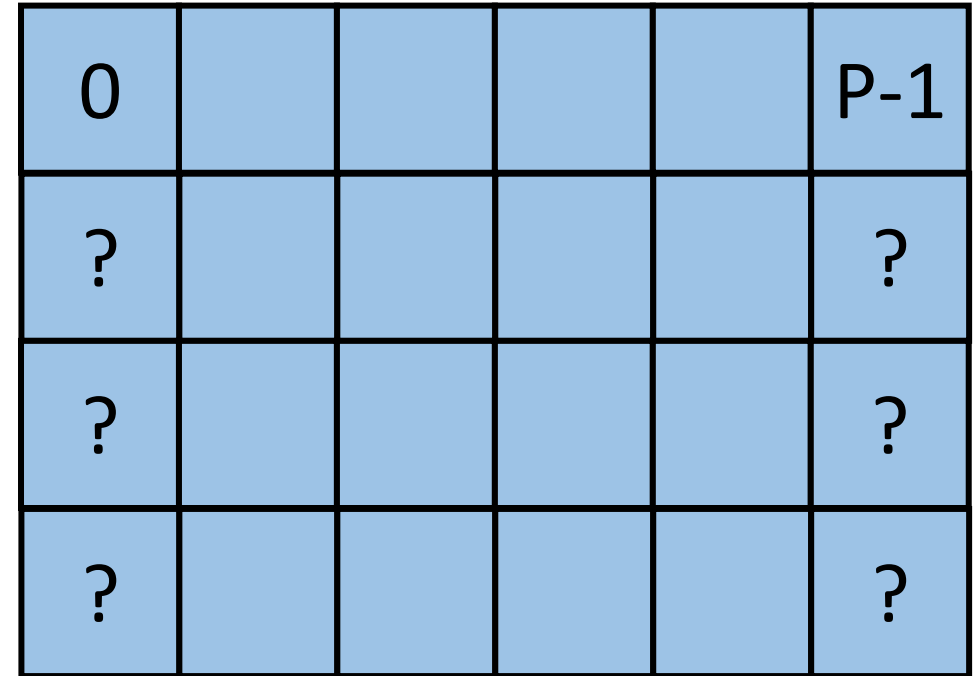
Domain



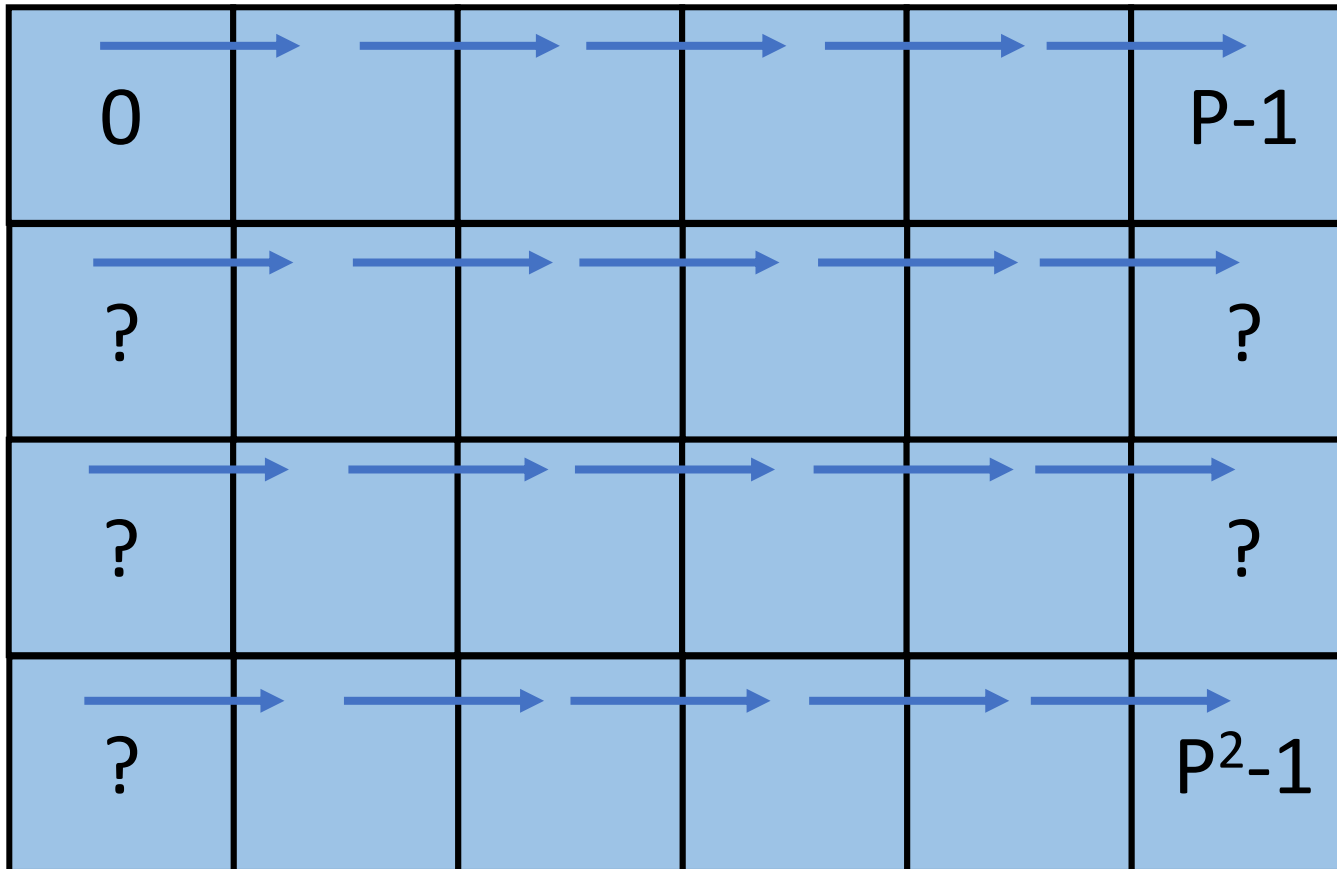
1D Domain



2D Domain



H.W.: 2D Domain 2-way Exchange in Each Row
(Assume P is a perfect square)



Communication Graph/Matrix for Communication Pattern on Slide 27 (P=9)

	512							
512		512						
	512							
				512				
			512		512			
				512				
							512	
						512		512
							512	

Assignment 1 – Use Blocking Send/Recv



Write a program to perform data (M bytes) exchange with two ranks that are D1 and D2 distances away on the right ($D1 < D2$). Assume that the total number of processes = P. If sender rank + D2 > P-1, then the sender sends only to D1. If sender rank + D1 > P-1, then the rank (i.e. invalid sender) does not send data to any rank. The receivers of a sender rank (D1 and D2 distances away) perform computations and send back the data to the sender.

The sequence for the sender ranks are communication(s) followed by computations at D1 and D2 followed by communication(s). This sequence of operations (you may select any order) occurs for T iterations.

Quiz 1

Question 1

"mpirun -np 100 -hosts host1:50,host2:50 ./parallelcode"

Type: Quiz Multichoice Questions Options:

Option 1: will run on 100 CPU cores

Option 2: will run on 100 processes

Option 3: will run on 2 nodes

Option 4: will give error

Question 2

```
if (myrank == 0) {  
    MPI_Send (data, 10, MPI_INT, 1, 20, MPI_COMM_WORLD);  
    MPI_Send (data, 10, MPI_INT, 1, 20, MPI_COMM_WORLD);  
}  
if (myrank == 1) {  
    MPI_Recv (data, 10, MPI_INT, 0, 20, MPI_COMM_WORLD, &status);  
    MPI_Recv (data, 10, MPI_INT, 0, 20, MPI_COMM_WORLD, &status);  
}
```

The above code snippet is not safe and it will not run successfully.

Question 3

```
// Assume BUFFER > 0 and there are even number of ranks
if (myrank % 2 == 0)
    MPI_Send(arr, BUFFER, MPI_INT, myrank+1, myrank, MPI_COMM_WORLD);
else
    MPI_Recv(recvarr, BUFFER, MPI_INT, myrank-1, myrank-1, MPI_COMM_WORLD,
    &status);
```

Option 1: The above code has tag mismatch

Option 2: The above code will run without any errors

Option 3: Odd ranks send BUFFER integers to even ranks

Option 4: Odd ranks receive BUFFER integers from even ranks

Question 4

```
// Assume BUFFER > 0 and there are even number of ranks
if (myrank % 2 == 0)
    MPI_Send(arr, BUFFER, MPI_INT, myrank+1, myrank+1, MPI_COMM_WORLD);
else
    MPI_Recv(recvarr, BUFFER, MPI_INT, myrank-1, myrank, MPI_COMM_WORLD,
    &status);
```

Option 1: The above code has tag mismatch

Option 2: The above code will run without any errors

Option 3: Odd ranks send BUFFER integers to even ranks

Option 4: Odd ranks receive BUFFER integers from even ranks

Question 5

// Assume that this code runs fine (i.e. there is a matching receive and there are no errors in the code).

// Assume $\text{BUFFER}/2 > \text{MPIR_CVAR_CH3_EAGER_MAX_MSG_SIZE}$

Code snippet 1: `MPI_Send(arr, BUFFER/2, MPI_INT, myrank+1, myrank, MPI_COMM_WORLD);`

Code snippet 2: `MPI_Send(arr, BUFFER, MPI_INT, myrank+1, myrank, MPI_COMM_WORLD);`

Option 1: Time taken by code snippet 1 is less than time taken by code snippet 2 (same execution environment, same senders/receivers)

Option 2: Time taken by code snippet 2 is less than time taken by code snippet 1 (same execution environment)

Question 6

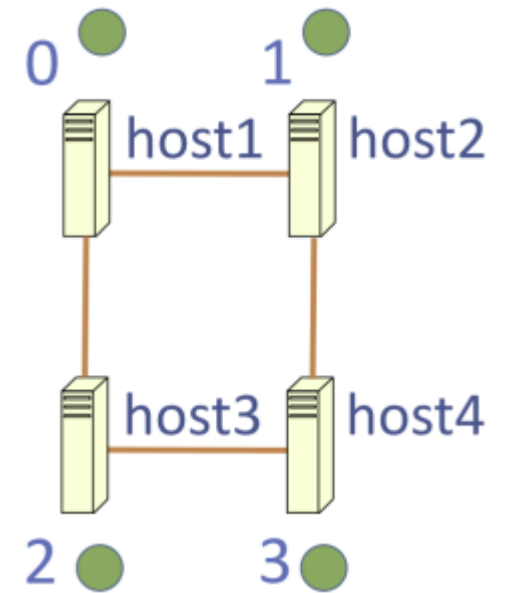
Consider the process placement as shown in the diagram on host1 to host4. Assume infinite network bandwidth.

Option 1: The maximum number of hops for the communication pattern $0 \rightarrow 3, 3 \rightarrow 0$ is 2

Option 2: The maximum number of hops for the communication pattern $0 \rightarrow 3, 3 \rightarrow 0$ is 4

Option 3: The average number of hops for the communication pattern $0 \rightarrow 3, 3 \rightarrow 0$ is 2

Option 4: The maximum number of hops for the communication pattern $0 \rightarrow 3, 3 \rightarrow 0$ is 3



Question 7

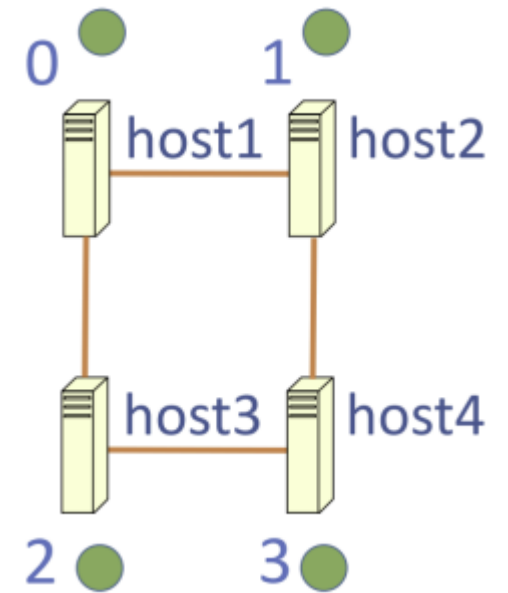
Consider the process placement as shown in the diagram on host1 to host4. Assume infinite network bandwidth.

Option 1: The maximum number of hops for the communication pattern $0 \rightarrow 1, 3 \rightarrow 2$ is 2

Option 2: The maximum number of hops for the communication pattern $0 \rightarrow 1, 3 \rightarrow 2$ is 1

Option 3: The maximum number of hops for the communication pattern $0 \rightarrow 3, 1 \rightarrow 2$ is 2

Option 4: The maximum number of hops for the communication pattern $0 \rightarrow 3, 1 \rightarrow 2$ is 3



Question 8

```
// size is the total number of ranks, BUFFER = 1 MB
if (myrank < size-1)
    MPI_Send(arr, BUFFER, MPI_INT, size-1, myrank, MPI_COMM_WORLD);
else {
    int count, recvarr[size][BUFFER];
    for (int i=0; i<size; i++)
        MPI_Recv(recvarr[i], BUFFER, MPI_INT, i, i, MPI_COMM_WORLD, &status);
}
```

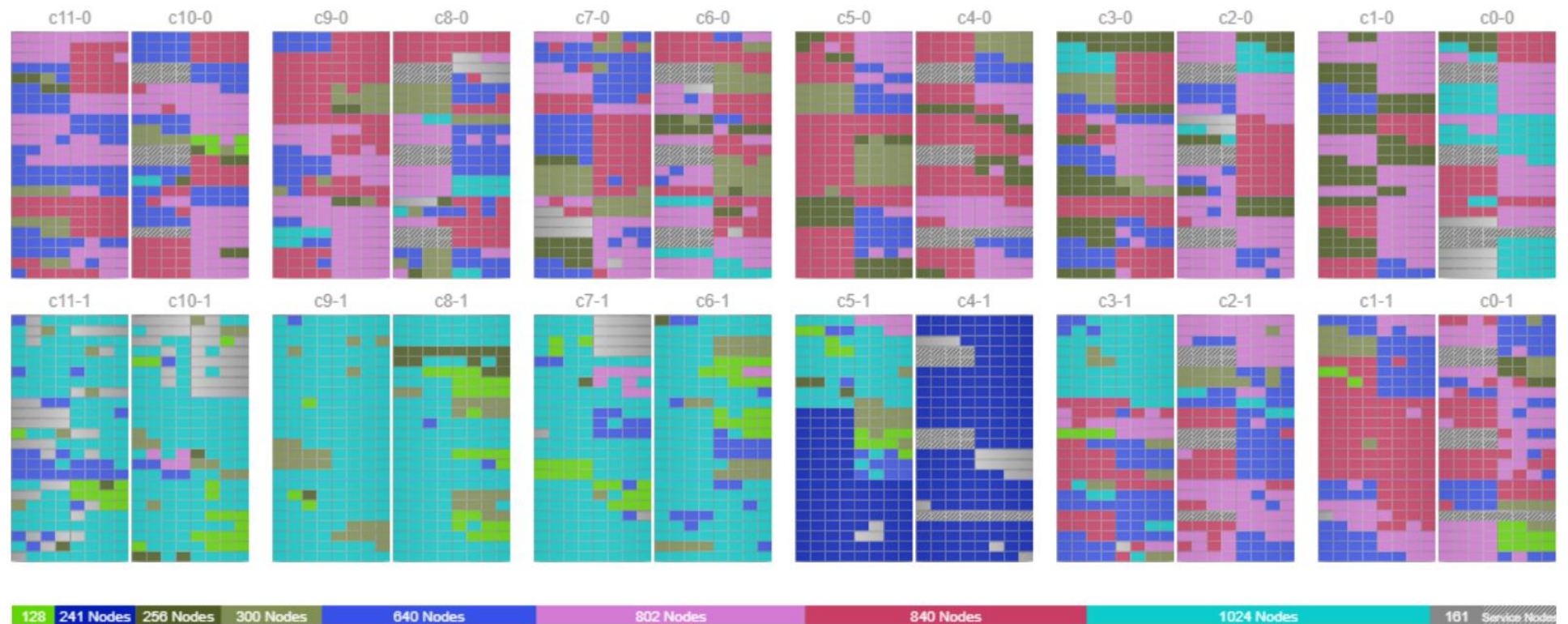
Option 1: All ranks send BUFFER integers to rank 0

Option 2: All ranks send BUFFER integers to the last rank

Option 3: All ranks receive BUFFER integers from the last rank

Option 4: All ranks receive BUFFER integers from rank 0

Supercomputer Job Allocation



Running

Starting

Queued

Reservations

Total Running Jobs: 8

Job Id	Project	Nodes	Start Time	Run Time	Walltime	Queue	Mode
512304	EstopSim_2	1024	9:13:30 AM	00:12:13	16:00:00	default	script
512623	TurbShockWalls	840	7:57:42 AM	01:28:01	1d 00:00:00	default	script
498557	TurbShockWalls	802	9:43:57 PM	11:41:46	1d 00:00:00	default	script
513358	PSFMat_2	640	8:29:27 AM	00:56:16	12:00:00	default	script
511830	ReconDepth	300	7:50:53 AM	01:34:50	06:00:00	default	script
514000	HighLumin	256	8:50:22 AM	00:35:21	06:00:00	default	script
514114	CVD_CityCOVID	241	1:28:47 AM	07:56:56	1d 12:00:00	CVD_Research	script
514178	FDTD_Cancer_2a	128	8:00:51 AM	01:24:52	03:00:00	default	script

status.alcf.anl.gov -> Theta (retired)

Resources Required

- Number of nodes
- Wall-clock time
- Users are charged for node-hours

Should there be any constraints on the above requirements?

User Jobs

- Different types of applications
- Interactive vs. batch jobs
 - Debug in interactive mode
- Exclusive vs. shared access
- Charged based on total resource usage
 - Job is killed when requested wall-clock time is over
 - Need to plan resource usage apriori

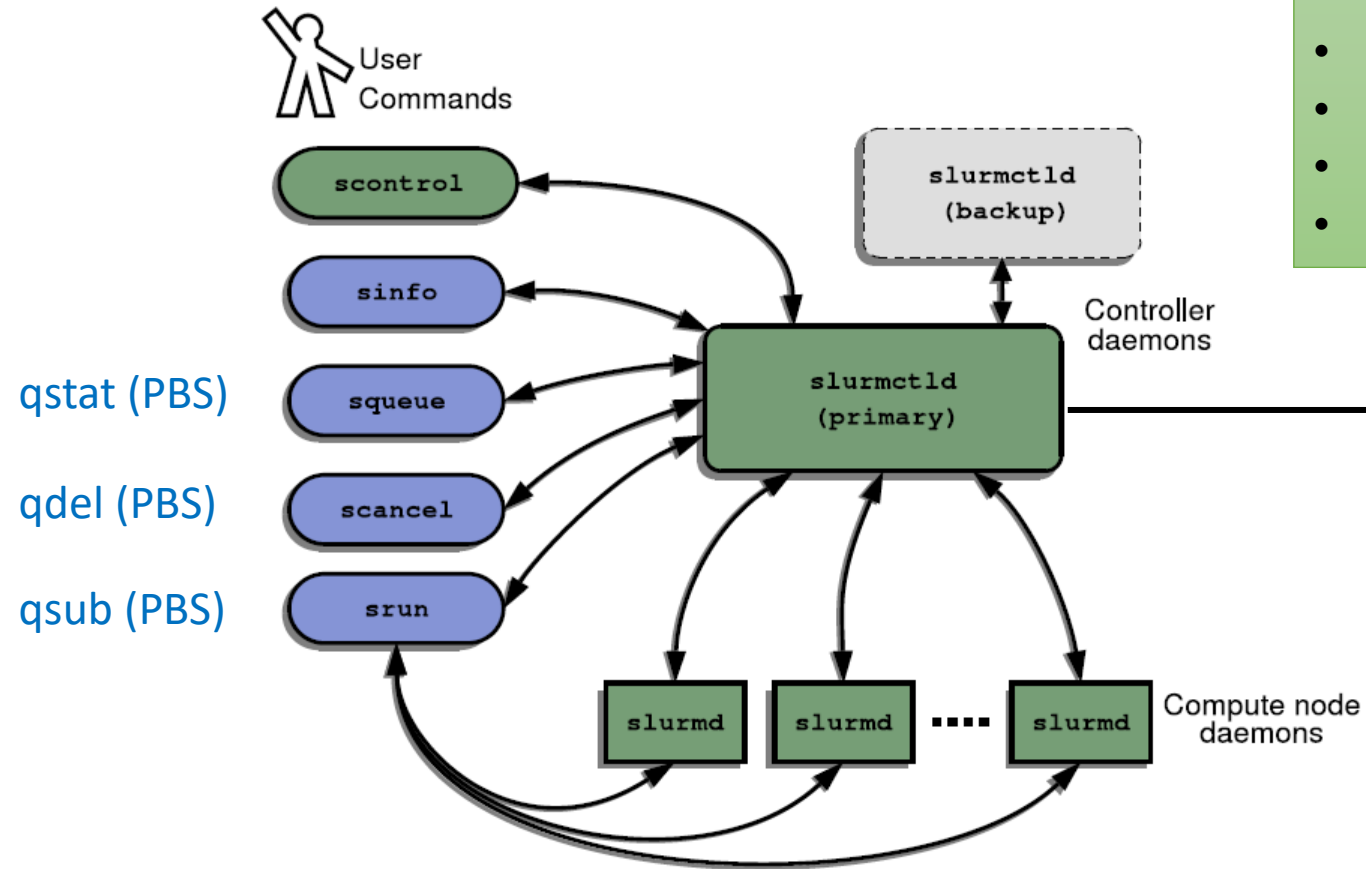
Batch Queueing Systems

- Schedules jobs based on queues
- Has full knowledge of queued, running jobs
- Has full knowledge of the resource usage
- Often combination of best fit, fair share, priority-based
- Designed to be generic, can be customized
- Suited to meet demands of the scheduling goals of the centre
- Typically FIFO/FCFS with backfilling

Workload managers/Schedulers

- Portable Batch System (PBS)
- LoadLeveler
- Application Level Placement Scheduler (ALPS)
- Moab/Torque
- Simple Linux Utility for Resource Management (SLURM)

SLURM



SLURM architecture [Jette et al.]

- Monitors states of nodes
- Accepts job requests
- Maintains queue of requests
- Schedules jobs
- Initiates job execution and cleanup
- Polls slurmd periodically
- Maintains complete state information

- Responds to controller requests
- Maintains job state
- Initiate, manage, cleanup processes
- I/O handling

SLURM Commands

```
#SBATCH -N 2
#SBATCH --ntasks-per-node=2
#SBATCH --error=job.%J.err
#SBATCH --output=job.%J.out
#SBATCH --time=00:10:00
#SBATCH --partition=standard
mpirun -np 4 ./exe <args>
```

Job Script

```
sbatch <jobscript>
queue
scancel <jobid>
```

queue

queue

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
12181	cpu	QE_FC	abc	R	1:21:59	16	rdcn[17-32]

queue --me

- Check the status of your job